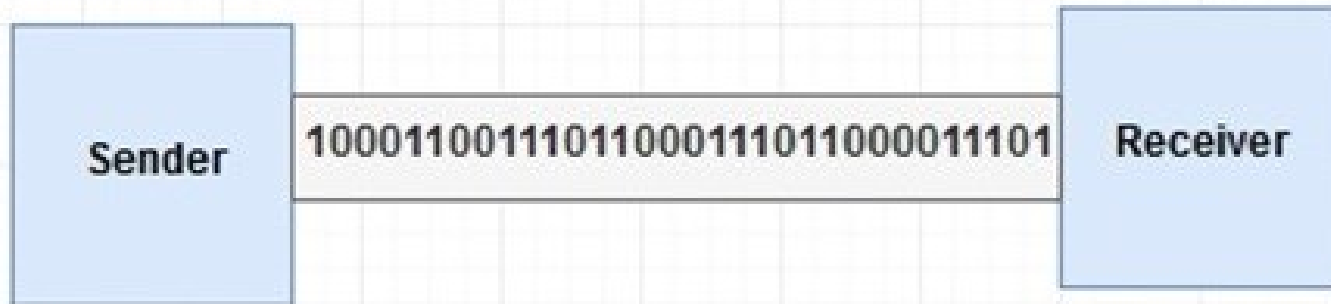


Serial Communication: UART, SPI, and I2C

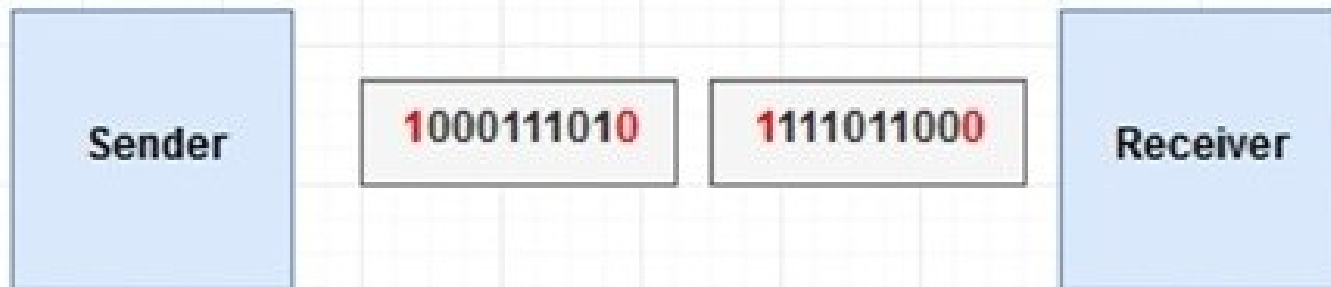
Serial Communication

- Synchronous
 - For synchronous data transfer, both the sender and receiver access the data according to **the same clock**.
 - SPI, I2C
- Asynchronous
 - For asynchronous data transfer, there is **no common clock** signal between the sender and receivers.
 - UART, **Controller Area Network (CAN bus)**

Synchronous Data Transfer



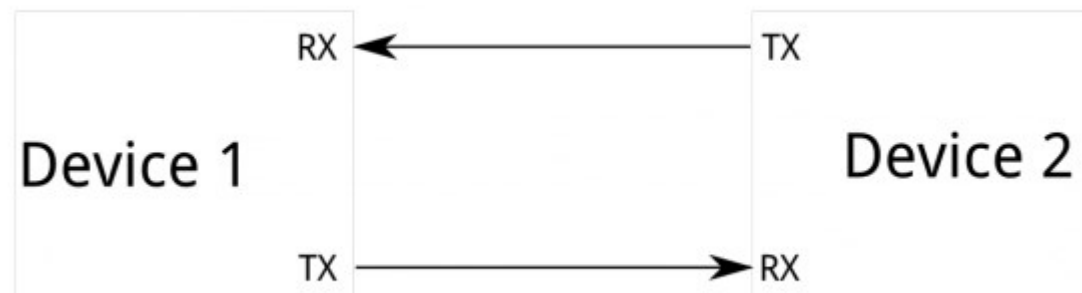
Asynchronous Data Transfer



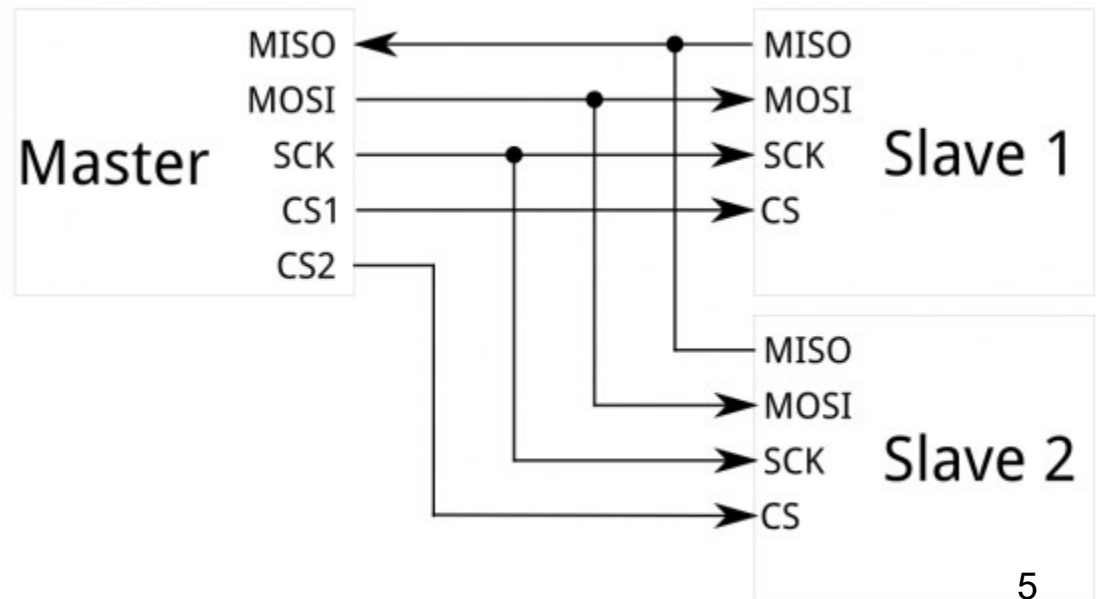
Data transferring direction

Why Use I2C?

- What's the disadvantages with Serial Ports?
 - UART(Universal Asynchronous Receiver and Transmitter)
 - Low data rate: At least **one start and stop bit** is a part of each frame of data, meaning that 10 bits of transmission time are required for each 8 bits of data sent, which eats into the data rate.
 - Point-to-point connections



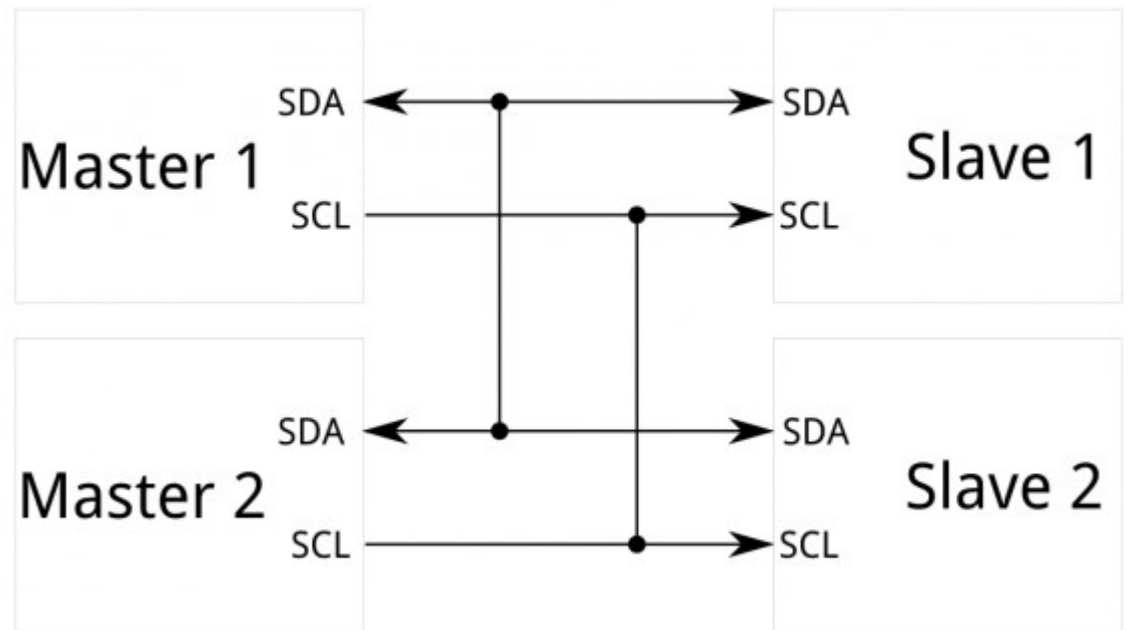
- What's the problems with SPI?
 - The most obvious drawback of SPI is **the number of pins required**.
 - Connecting a single master to a single slave with an SPI bus requires four lines; each additional slave requires one additional chip select I/O pin on the master.
- SPI is a bus.
 - One to many connections
 - Data lines:
 - MISO: master in slave out
 - MOSI: master out slave in
 - Clock: SCK
 - Chip Select: CS1, CS2, ...



I2C - The Best of Both Worlds!

- Inter-IC Communication (I2C)
 - A multi-master Bus
 - Use Only Two lines (SCL, SDA) as well as the ground line.

- There is some overhead with I²C; for every 8 bits of data to be sent, one extra bit of meta data (the “ACK/NACK” bit) must be transmitted.
- The hardware required to implement I²C is more complex than SPI, but less than asynchronous serial.
I2C can be fairly trivially implemented in software.



I²C History

- I²C was originally developed in 1982 by Philips for various Philips chips.
 - The original spec allowed for only 100kHz communications, and provided only for 7-bit addresses,
 - limiting the number of devices on the bus to 112 (there are several reserved addresses, which will never be used for valid I²C addresses).
 - In 1992, the first public specification was published, adding
 - a 400kHz **fast-mode** as well as an expanded 10-bit address space.
 - Much of the time (for instance, in the ATMega328 device on many Arduino-compatible boards) , device support for I²C ends at this point.
 - There are three additional modes specified:
 - fast-mode plus, at 1MHz;
 - high-speed mode, at 3.4MHz; and
 - ultra-fast mode, at 5MHz.

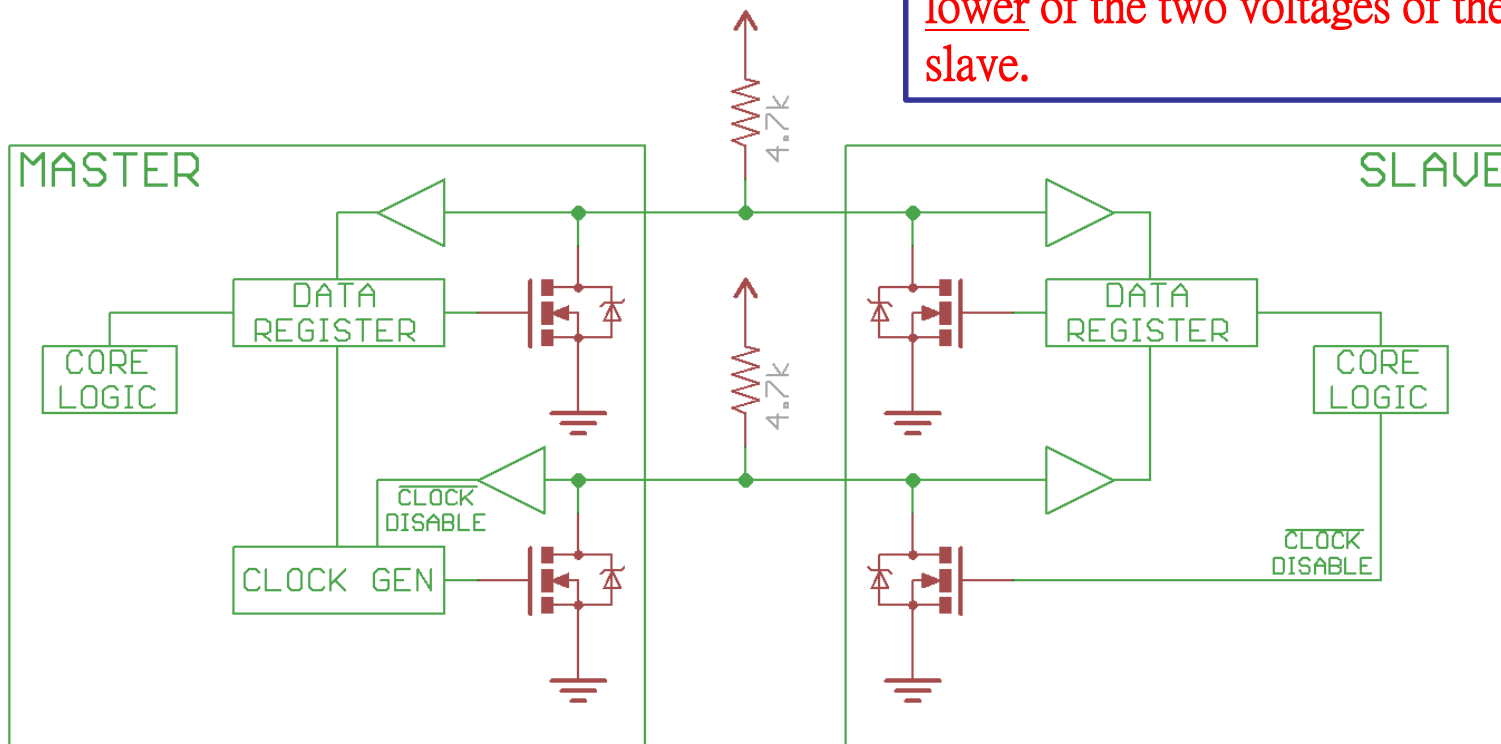
I2C at the Hardware Level Signals

- Each I²C bus consists of two signals:
 - SCL and SDA.
 - SCL is the clock signal, and
 - SDA is the data signal.
 - The clock signal is always generated by the current bus master;
 - some **slave devices may force the clock low at times** to delay the master sending more data (or to require more time to prepare data before the master attempts to clock it out). This is called “clock stretching”.

Open drain – wired-AND logic

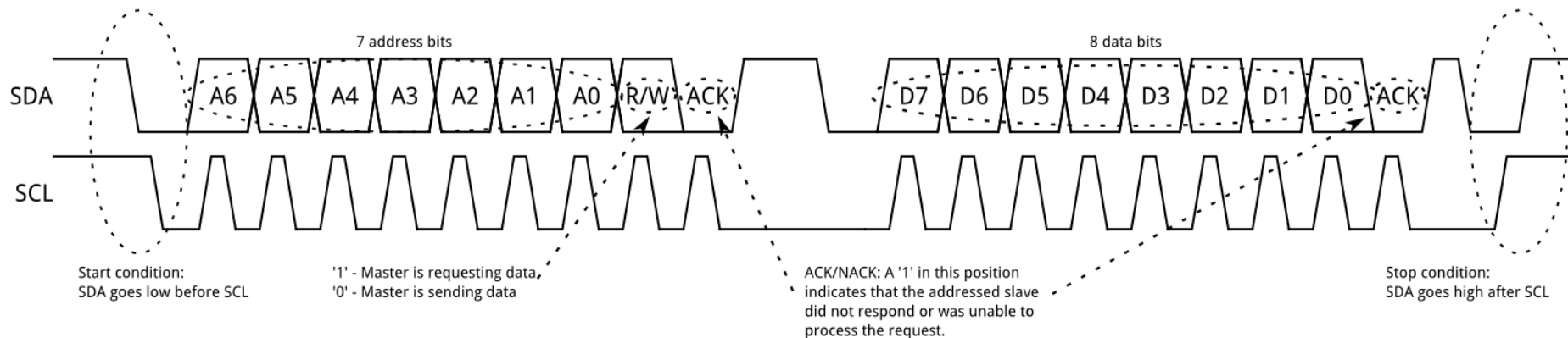
- Unlike UART or SPI connections, the I²C bus drivers are “open drain”,
 - meaning that they can pull the corresponding signal line low, but cannot drive it high. → **Wired-AND logic**
 - Each signal line has a common pull-up resistor on it, to restore the signal to high when no device is asserting it low.

The trick is to connect the pull-up resistors to the lower of the two voltages of the master and the slave.



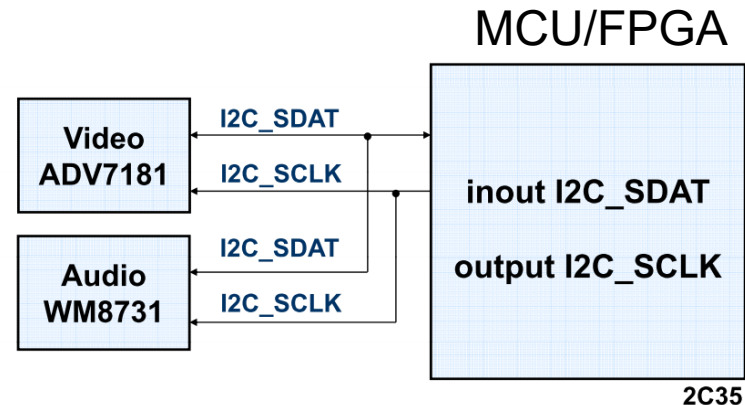
I2C Protocol

- Start Condition
 - To initiate the address frame, the master device leaves SCL high and pulls SDA low.
- Address Frame
 - The address frame is always first in any new communication sequence. For a 7-bit address, the address is clocked out most significant bit (MSB) first, followed by a R/W bit indicating whether this is a read (1) or write (0) operation
- Data Frame
- Stop Condition



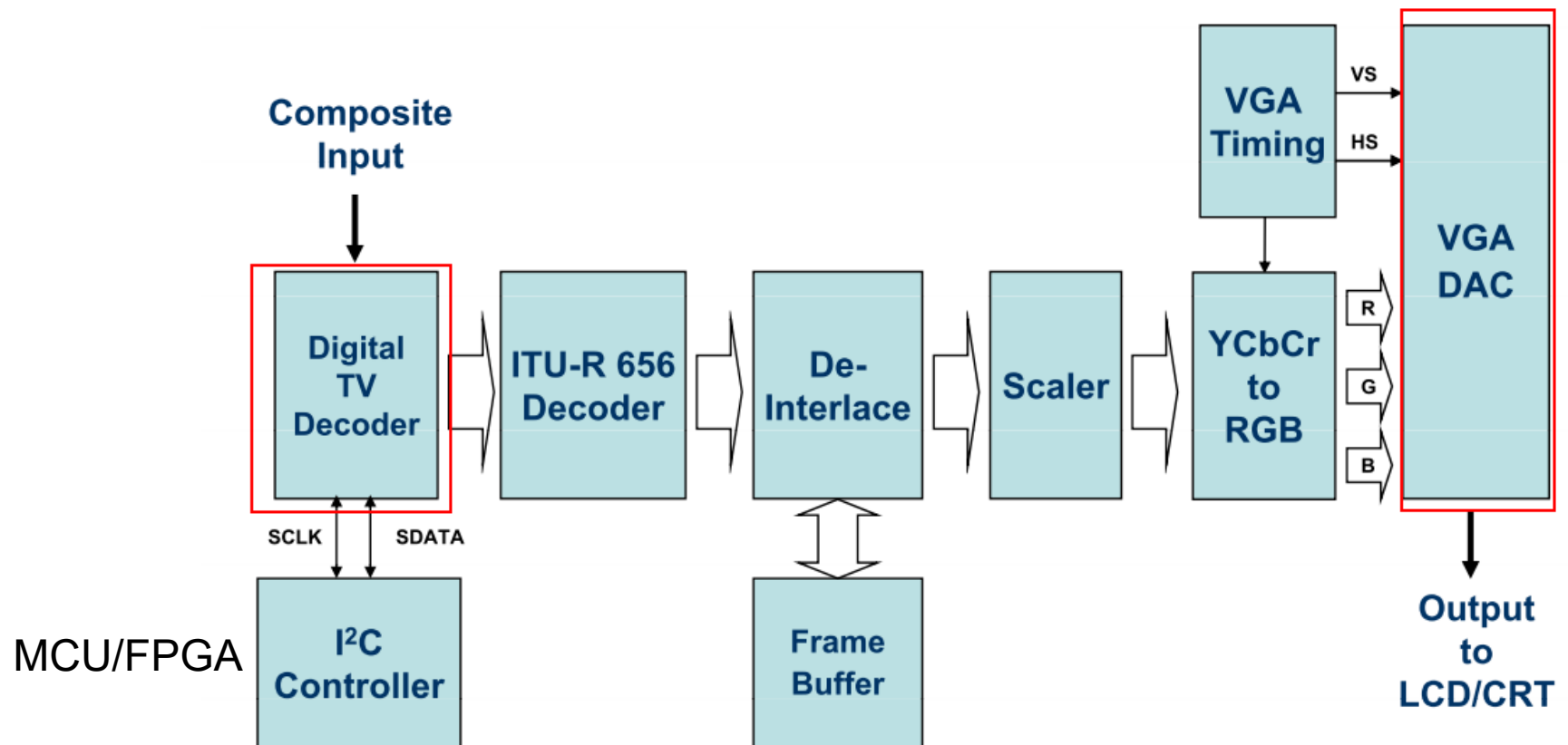
Applications of I2C

- Computer main boards
- TV main boards
- Printer
- Digital Camera
- ...



A Digital TV main board

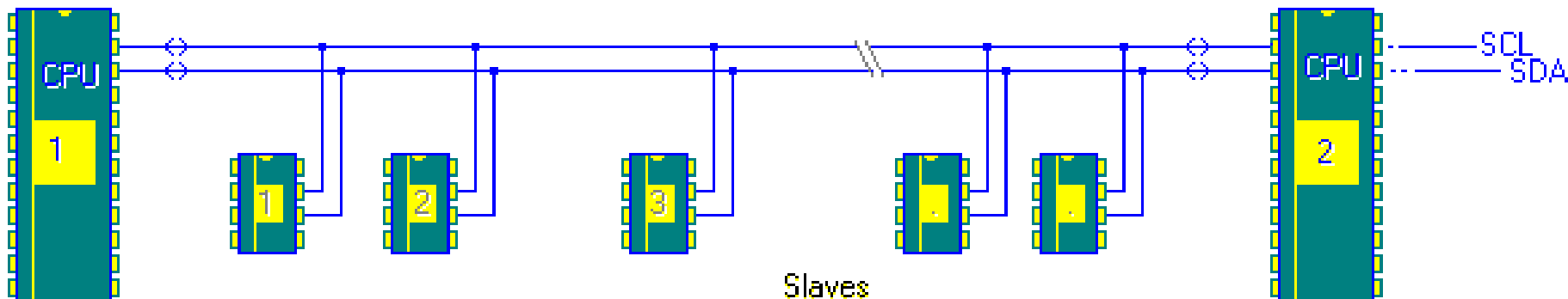
- The red blocks are ICs.
- The other blocks might be implemented in a microprocessor or FPGA



I2C Arbitration

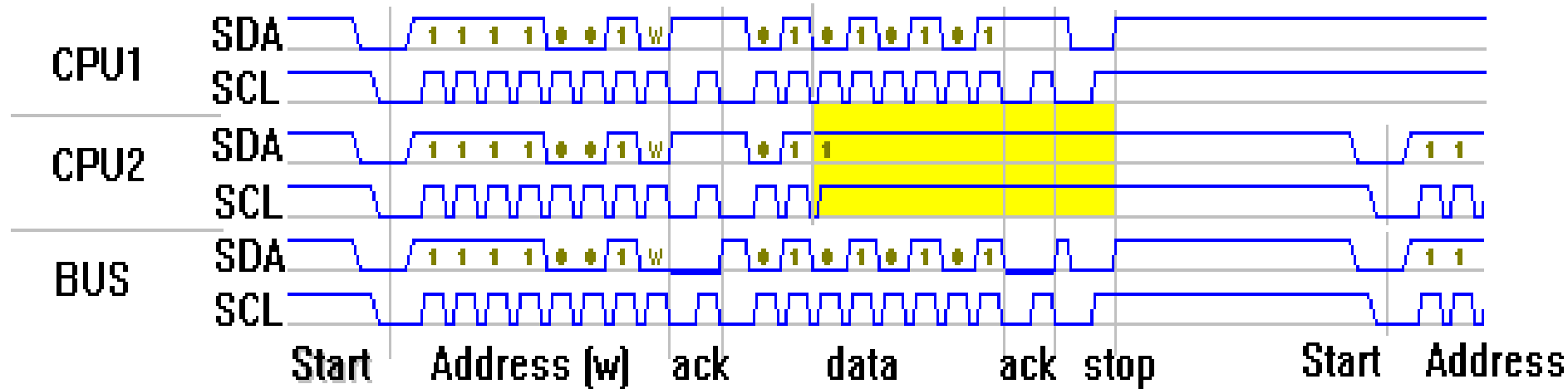
I2C Arbitration

- A master loses arbitration when it cannot get either SCL or SDA to go high
- SCL and SDA are wired-AND signals



Arbitration example

- CPU2 loses arbitration
- <http://www.esacademy.com/faq/i2c/index.htm>



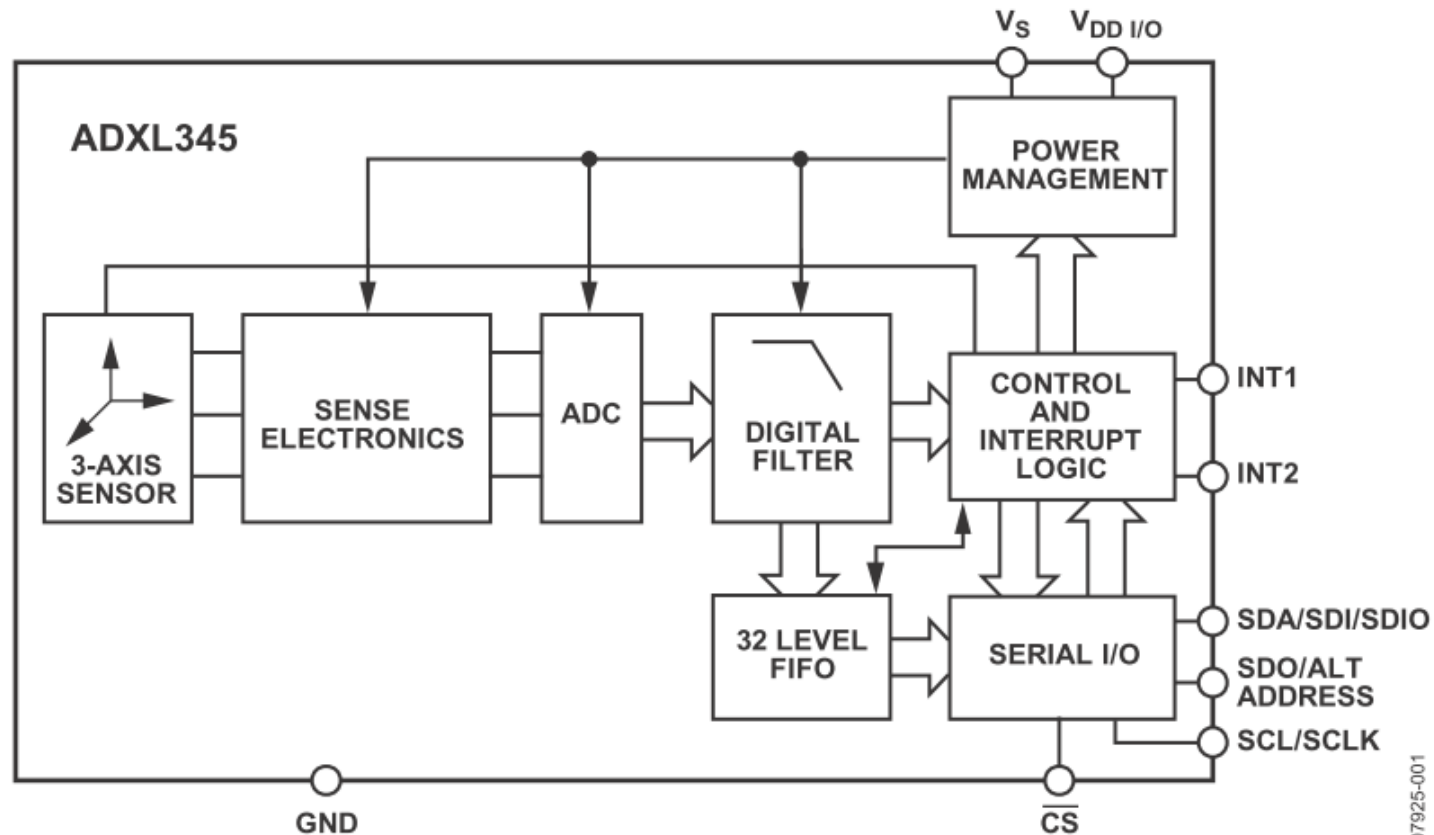
I2C synchronization mechanism

- Slow slaves not ready
 - The slave that wants the master to wait simply pulls the SCL low as long as needed. This is like adding "wait states" to the I2C bus cycle.
 - You can make this technique rigid by not pulling only the SCL line low, but also the SDA line.

I2C sensor example

ADXL 345: a 3D Accelerator sensor

FUNCTIONAL BLOCK DIAGRAM



I2C Connection: RPi with the sensor ADXL 345

- I2C Connection (from ADXL 345 data sheet)– With CS tied high to VDD I/O, the ADXL345 is in I2C mode

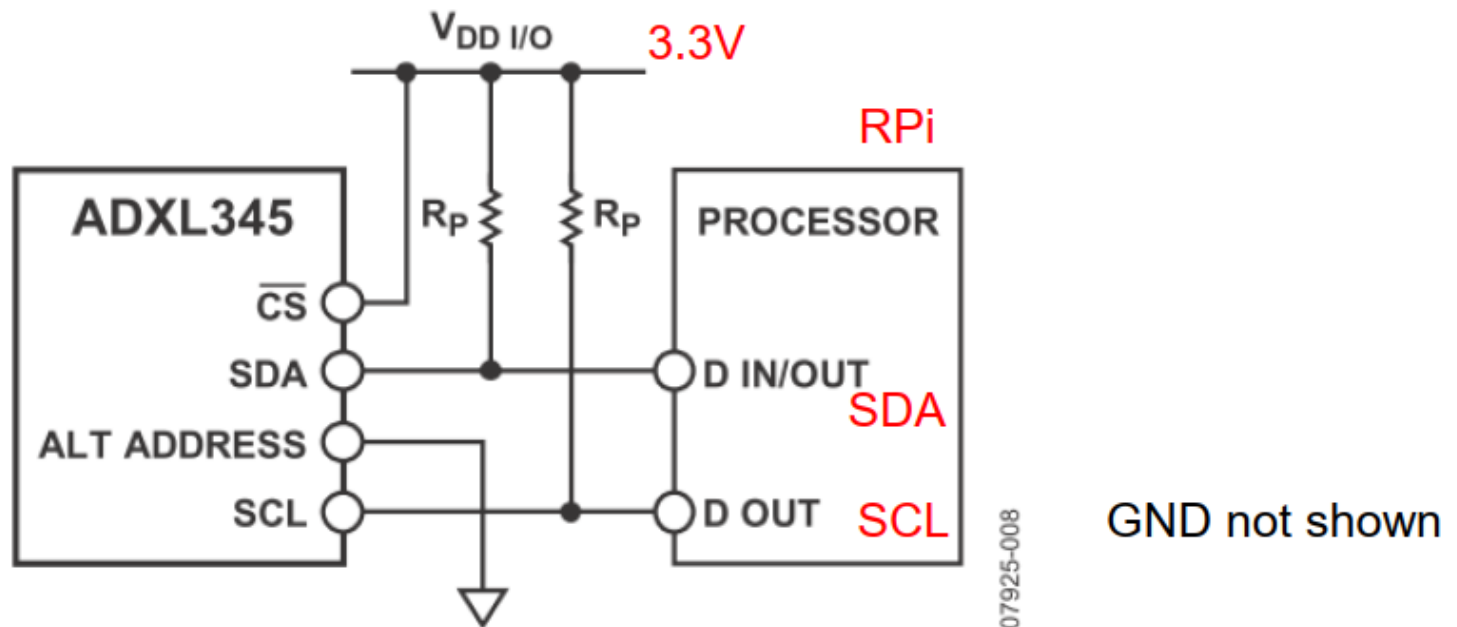


Figure 40. I²C Connection Diagram (Address 0x53)

Reference code for RPi to read sensor ADXL345

- I2C mode
 - <https://roboticsbackend.com/wiringpi-i2c-tutorial-raspberry-pi-adxl345/>
- SPI mode
 - <https://github.com/nagimov/adxl345spi>

Review Questions

- What are the main differences I2C between SMBus (System Management Bus)?
- What is the I2C address of ADXL 345, if ALT ADDRESS is connected to HIGH?
 - Hint: check the ADXL345 manual
- How to connect two open-collector (or open-drain) signal lines to achieve the wired-AND logic?
- What is the main difference between the bus master and the bus slave?